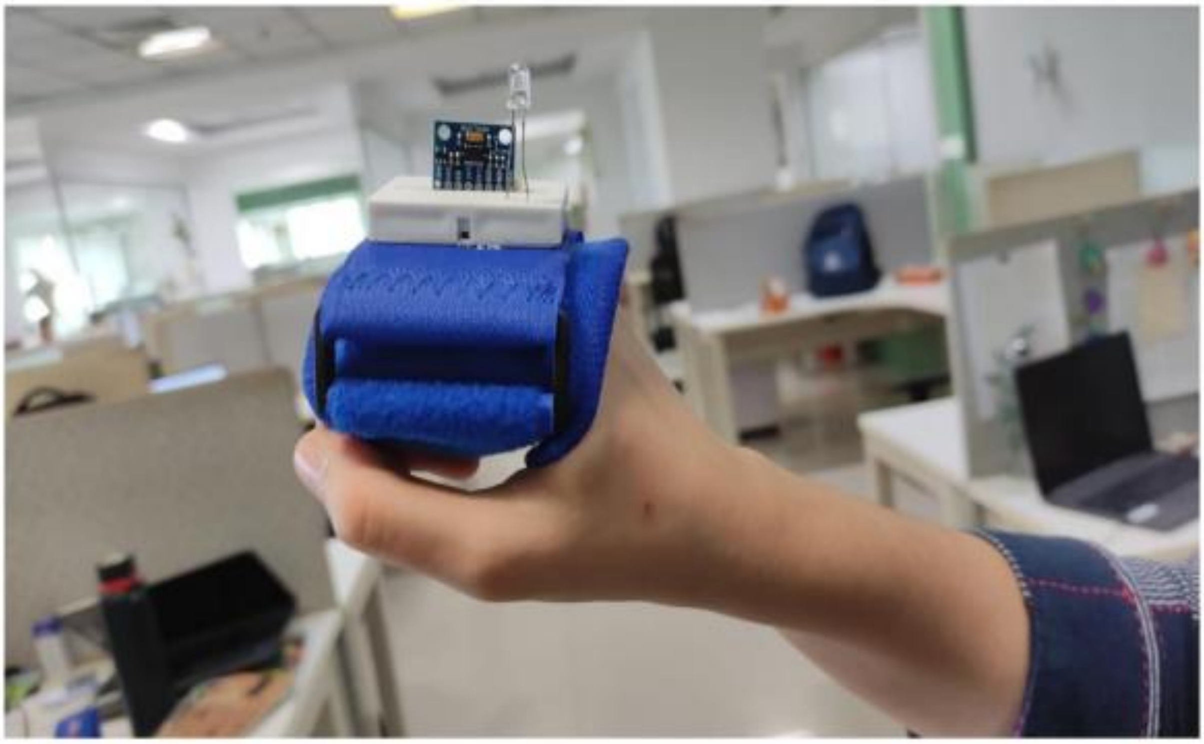
Power Gloves(Build Gesture-controlled Mouse)

Introduction to the course:	Build hand gesture-based devices that can be used as alternate control devices for PC mouse or to navigate a mobility device.
What does this course aim to achieve?	• Learn about alternate PC control • Learn about inertia measurement sensors, HID devices, and how to convert a normal microcontroller to a PC HID device. • Learn more about the people with limited hand movements and their requirements. • Learn about Bluetooth wireless communication • Learn about IMU sensors, Motor controller boards, and Bluetooth modules. • Learn about AT commands and configuration • Learn how gestures can be converted to mobility
What is being built in this course:	1) A wearable gesture-controlled mouse that helps to access a Windows PC without a mouse/keyboard. The microcontroller mimics an HID device and does the mouse movements. 2) A wearable hand gesture device that includes a gyro sensor & microcontroller, which converts gyro values to corresponding motor movements and sends the data to an RC car via Bluetooth
How is it being tested:	1) Different people will have different comfortable hand positions; the device can adjust to that position with the click of a calibration switch. Activities are given to adjust sensitivity values and also to look for better positions for the sensor and right-click/left-click buttons 2) The Controllability of the RC car can be corrected by changing the sensitivity value as per individual preference. If the movements are beyond the possible correction values, the sensors/Car setup/wiring needs to be reconfigured/tested
Course Prerequisites	• Basic C/C++ programming skill • Basic understanding of sensors and other electronics • Build a club project on maze solving robot

Build a Gesture controlled Mouse for PC



Immersive Technology Entrepreneurship Labs

Index

Prerequisite

Topic	Resources
Understanding Gyro Sensor & HID	HID
AT Command	Link

Outcome

Build a hand gesture-based device for people with limited hand motor movements that can control the mouse pointer of a PC.

Concept

In this project, we will create a gesture-based device that mimics mouse actions and controls a Windows PC. This device can be used by persons with hand impairments who cannot use a conventional mouse to control the PC. The wearable gesture device could communicate with the PC which can in turn communicate with apps and avail many services such as communication software or accessing the internet for healthcare, food delivery, etc.

Here we will be using an ESP32 module as an HID device.

HID (Human Interface Device)

A human interface device, often known as a HID, is a kind of computer hardware that is typically used by people and that receives input from people and provides outputs to people in many forms.

The USB-HID specification is most frequently used when the term "HID" is used. Mouse, Keyboards & Joysticks are all part of HID Devices.

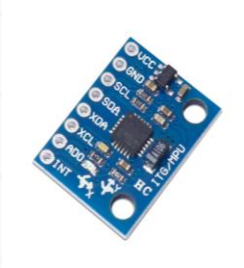
Components

Sno	Components with specification	Quantity
1.	ESP32 C6 Dev Module	1
2.	USB to Type – C cable	1
4.	IMU - MPU6050 Sensor	1
5.	Small Breadboard (170 Points)	1
6.	Button module	2
7.	Long Wires	1
8.	Jumper Wires	As needed

9.	Wrist Band	1
10.	Processing Software	1

Note: All the components above are reusable for other projects

Esp 32 C-6 Module



MPU6050 sensor

Push Button

Jumper Wires

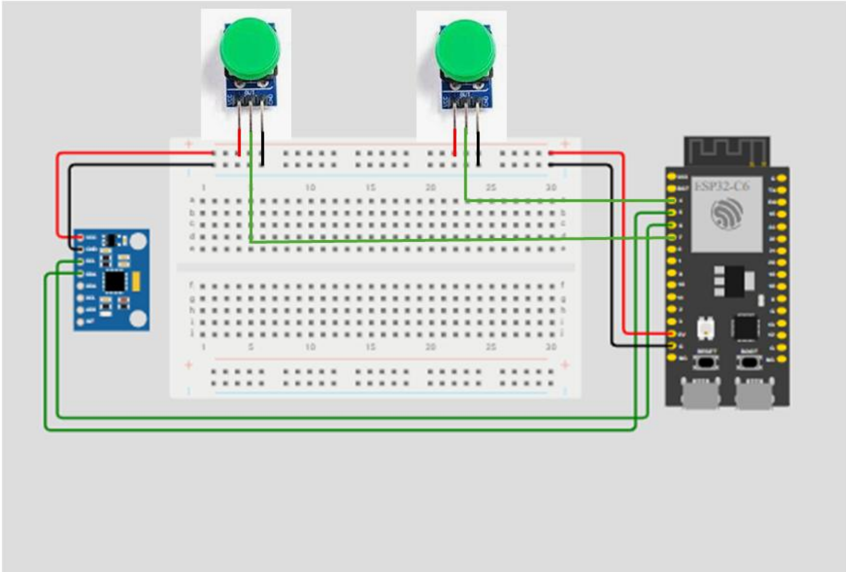
USB

Breadboard

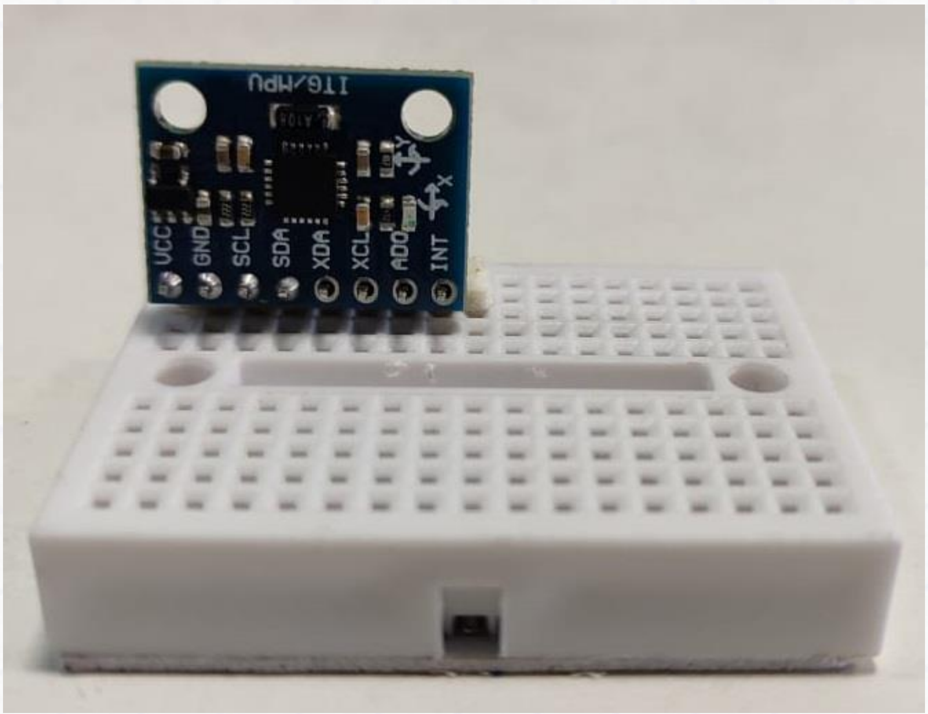
● NOTE: Before starting the connections, verify using a multimeter that all the wires are working. Also ensure that the connections are strong, else the setup may not work.

Hardware Setup - Connections

Circuit Diagram



Orientation of MPU6050 sensor



Since we are using only y and z axis values for this project. Orient the sensor in such a way that it is perpendicular to the breadboard as shown in the above picture.

Detailed Connection Steps

Stick the small breadboard to the wristband you have in this orientation

MPU6050 sensor

(MPU to ESP32)

- VCC to V5
- GND to GND
- SCL to G6
- SDA to G5

Button module

(Buttons to ESP32)

Left Button

- VCC to V5
- GND to GND
- OUT to G7

Right Button

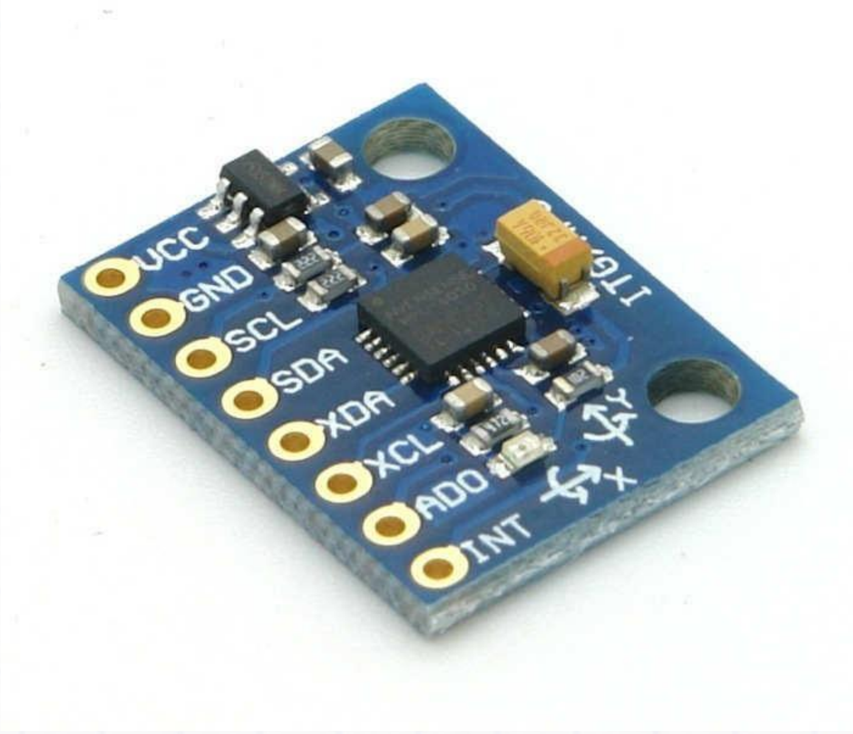
- VCC to V5
- GND to GND
- OUT to G9

Install Required Libraries

- Go to Sketch > Include Library > Manage Libraries.
- Connect Your ESP32 C-6 Module
- Connect your ESP32 Module to your computer using a USB cable.
- Go to Tools > Board > ESP32C6 Dev Module.
- Go to Tools > Port and select the COM port to which your ESP32C-6 is connected.
- Go to sketch > Include Library > Manage Libraries.
- Search MPU6050 and install Adafruit MPU6050 by Adafruit.
- There is a file attached with the manual, extract it and copy paste into Documents > Arduino > Libraries.

Components in detail

1) MPU6050 Sensor Module



MPU6050 sensor module is complete 6-axis Motion Tracking Device. It combines 3-axis Gyroscope, 3-axis Accelerometer and Digital Motion Processor all in small package.

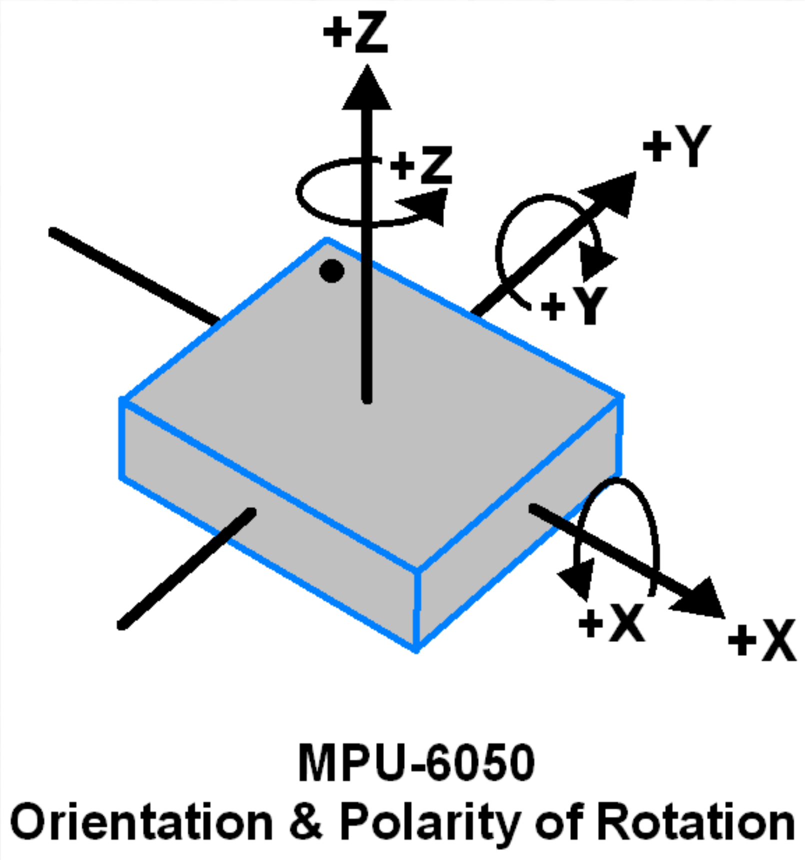
NOTE: Also, the sensor has an additional feature of on-chip Temperature sensor. It has I2C bus interface to communicate with the microcontrollers.

It has Auxiliary I2C bus to communicate with other sensor devices like 3-axis Magnetometer, Pressure sensor etc.

If the 3-axis Magnetometer is connected to the auxiliary I2C bus, then the MPU6050 can provide complete 9-axis Motion Fusion output.

3-Axis Gyroscope

The MPU6050 consists of a 3-axis Gyroscope with Micro Electromechanical System (MEMS) technology. It is used to detect rotational velocity along the X, Y, Z axes as shown in below figure.



When the gyros are rotated about any of the sense axes, the Coriolis Effect causes a vibration that is detected by a MEM inside MPU6050.

The resulting signal is amplified, demodulated, and filtered to produce a voltage that is proportional to the angular rate.

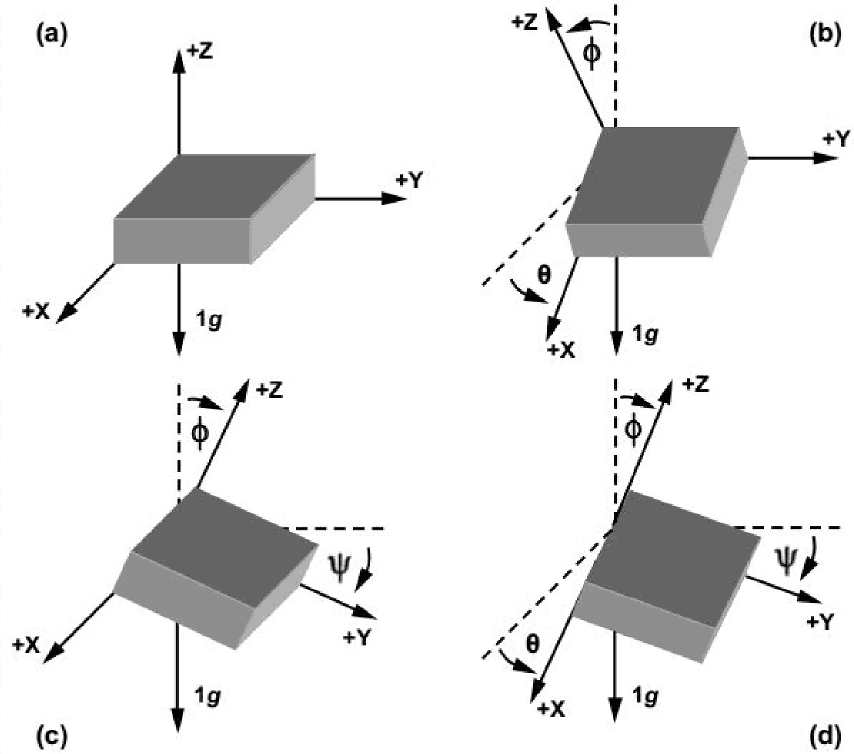
This voltage is digitized using 16-bit ADC to sample each axis.

The full-scale range of output are +/- 250, +/- 500, +/- 1000, +/- 2000.

It measures the angular velocity along each axis in degrees per second unit.

3-Axis Accelerometer

The MPU6050 consists of a 3-axis Accelerometer with Micro Electromechanical (MEMs) technology. It is used to detect angle of tilt or inclination along the X, Y and Z axes as shown in below figure.



Acceleration along the axes deflects the movable mass.

This displacement of moving plate (mass) unbalances the differential capacitor which results in sensor output. Output amplitude is proportional to acceleration.

16-bit ADC is used to get digitized output.

The full-scale range of acceleration are +/- 2g, +/- 4g, +/- 8g, +/- 16g.

It is measured in g (gravity force) units.

When the device is placed on a flat surface it will measure 0g on the X and Y axis and +1g on the Z axis.

The MPU-6050 module has 8 pins as listed below:

1. **INT**: Interrupt digital output pin.

2. **AD0**: I2C Slave Address LSB pin. This is 0th bit in 7-bit slave

address of device. If connected to VCC then it is read as logic

one and slave address changes.

3. **XCL**: Auxiliary Serial Clock pin. This pin is used to connect other I2C interface enabled sensor SCL pin to MPU-6050.

4. **XDA**: Auxiliary Serial Data pin. This pin is used to connect other I2C interface enabled sensor SDA pin to MPU-6050.

5. **SCL**: Serial Clock pin. This pin to connected to microcontrollers SCL pin.

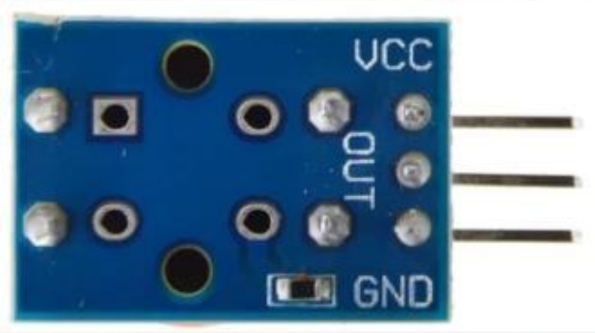
6. **SDA**: Serial Data pin. This pin is connected to microcontrollers SDA pin.

7. **GND**: Ground pin. This requires a ground connection.

8. **VCC**: Power supply pin. This requires a +5V DC supply.

For detailed theory refer to this Link

2) Button module



Features

- Interface: Standard electronic block (GND, VCC, SIG)

- Working voltage: 2 - 5.5 Volt DC
- Working Current: 0.55mA (MAX)
- Output: Digital level (press high, release low)

Software - Implementing the gesture project

Downloads & Installation

Download the project zip file from the build club drive and unzip it on your computer.

Steps to install libraries

1. Unzip the *.zip file
2. Go to the extracted folder > libraries
3. Copy the ***MPU6050*** and ***ESP32_BLE_Mouse*** folder under the libraries folder.
4. Paste it under your **Documents > Arduino > Libraries**
5. Use this link to download processing software.

   Note: While downloading please cross check the software version should be (4.4.7).

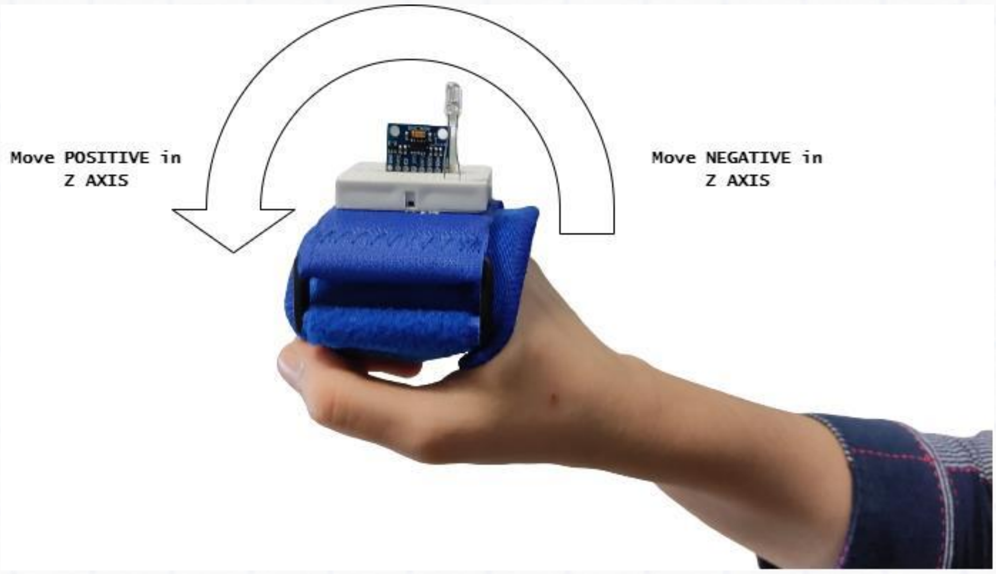
Implementing the Code

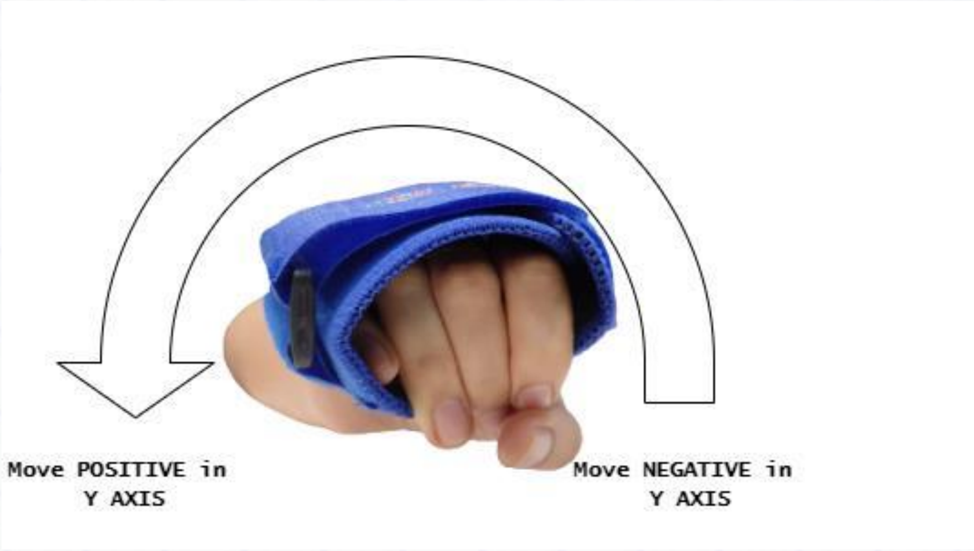
The ***mouse.ino*** turns an ESP32 device with an MPU6050 sensor into a Bluetooth Low Energy (BLE) mouse. It uses gyroscope data from the MPU6050 to control cursor movement and includes functionality for left and right mouse clicks using physical buttons connected to the ESP32.

```
#include <Wire.h>
#include <MPU6050.h>
#include <math.h>

MPU6050 mpu;

int16_t ax, ay, az;
int16_t gx, gy, gz;

#define LEFT_BTN 7
#define RIGHT_BTN 4

void setup() {
  Serial.begin(115200);
  Wire.begin(5, 6);   // SDA=GPIO5, SCL=GPIO6

  pinMode(LEFT_BTN, INPUT_PULLUP);
  pinMode(RIGHT_BTN, INPUT_PULLUP);

  mpu.initialize();
  if (mpu.testConnection()) {
    Serial.println("MPU6050 connected!");
  } else {
    Serial.println("MPU6050 connection failed!");
  }
}

void loop() {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  // Convert raw data to tilt angles
  float angleX = atan2(ay, az) * 180.0 / PI;   // Roll (left/right)
  float angleY = atan2(-ax, sqrt((long)ay * ay + (long)az * az)) * 180.0 / PI;
// Pitch (forward/back)

  int left = digitalRead(LEFT_BTN) == LOW ? 1 : 0;
  int right = digitalRead(RIGHT_BTN) == LOW ? 1 : 0;

  // Send data: angleX,angleY,left,right
  Serial.print(angleX, 2);
  Serial.print(",");
  Serial.print(angleY, 2);
  Serial.print(",");
  Serial.print(left);
  Serial.print(",");
```

```
  Serial.println(right);

  delay(50);
}
```

Note: The above code to be uploaded in ESP32C6 Dev board using Arduino IDE to read the angle.





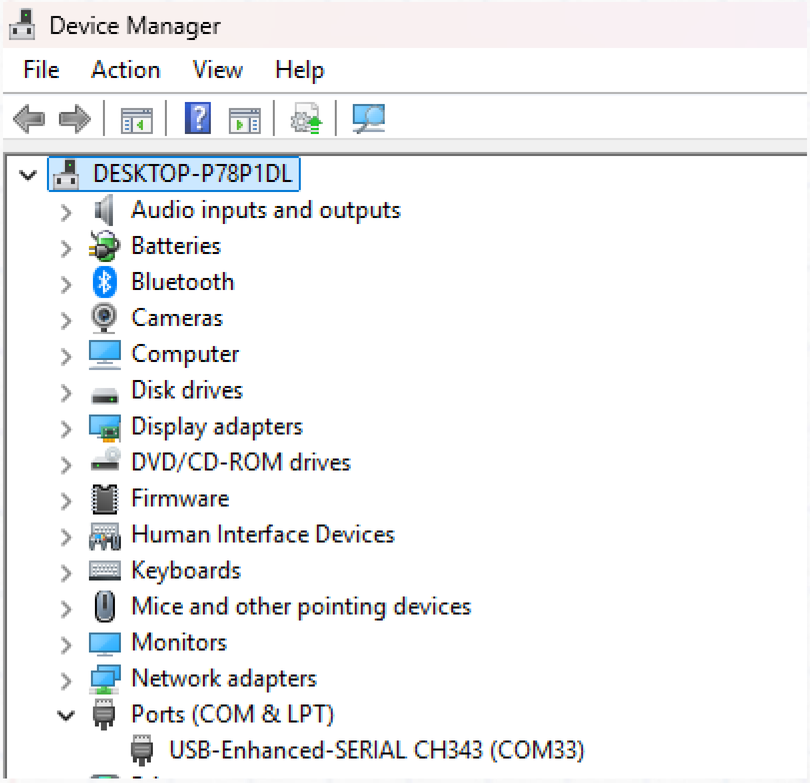
```
import processing.serial.*;

import java.awt.Robot;

import java.awt.AWTException;

import java.awt.MouseInfo;

import java.awt.event.InputEvent;

Serial myPort;

Robot robot;

float xpos, ypos;

float sensitivity = 0.2; // adjust mouse speed

void setup() {

  size(400, 300);

  background(200);

  // Open COMXX

  myPort = new Serial(this, "COMXX", 115200);

  myPort.bufferUntil('\n');

  try {

    robot = new Robot();

  } catch (AWTException e) {

    e.printStackTrace();

  }
```

```
  xpos = MouseInfo.getPointerInfo().getLocation().x;

  ypos = MouseInfo.getPointerInfo().getLocation().y;

}

void draw() { }

void serialEvent(Serial myPort) {

  String inString = myPort.readStringUntil('\n');

  if (inString != null) {

    inString = trim(inString);

    String[] values = split(inString, ',');

    if (values.length == 4) {

      float angleX = float(values[0]); // left/right

      float angleY = float(values[1]); // forward/back

      int left = int(values[2]);

      int right = int(values[3]);

   // Move mouse based on tilt angles

      xpos += map(angleX, -45, 45, -10, 10) * sensitivity;

      ypos += map(angleY, -45, 45, -10, 10) * sensitivity;

      xpos = constrain(xpos, 0, displayWidth-1);

      ypos = constrain(ypos, 0, displayHeight-1);
```

```
      robot.mouseMove(int(xpos), int(ypos));

      // Left Click

      if (left == 1) {

        robot.mousePress(InputEvent.BUTTON1_DOWN_MASK);

        robot.mouseRelease(InputEvent.BUTTON1_DOWN_MASK);

      }

      // Right Click

      if (right == 1) {

        robot.mousePress(InputEvent.BUTTON3_DOWN_MASK);

        robot.mouseRelease(InputEvent.BUTTON3_DOWN_MASK);

      }

    }

  }

}
```

Note: The above code to be uploaded in Processing Software, while uploading please close the Serial Monitor in Arduino IDE, then only the project works.

- While uploading please verify the COM port in your laptop and mention that COM port in the above code.



Tasks

1.Try changing the sensitivity value and find out a value suitable for each of your teammates. Try opening multiple apps in PC

2. Interchange the sensor and button positions and check whether the sensitivity is suitable for people with limited hand movements.

3. Try to position the sensors and buttons, so that it can be operated by single hand.

4. Make a video presenting the build of a gesture device, how you completed the activities and challenges. Share it with the Build Club Community on the Discord Server!

Club Activity:

1)

2)

Gesture Control Visuals

Software – implementing the code

```
// ESP32-C6 MPU6050 raw reader
// SDA = GPIO5, SCL = GPIO6 (change if you used other pins)
#include <Wire.h>
const int SDA_PIN = 5;
const int SCL_PIN = 6;
const uint8_t MPU_ADDR = 0x68; // if AD0 HIGH use 0x69
void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN, SCL_PIN);
  delay(100);
  // wake up MPU6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B); // PWR_MGMT_1
  Wire.write(0x00); // wake
  Wire.endTransmission();
  delay(50);
  Serial.println("MPU6050 init done");
  // read WHO_AM_I for debug
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x75);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, (uint8_t)1);
  if (Wire.available()) {
    uint8_t who = Wire.read();
```

```
    Serial.print("WHO_AM_I = 0x"); Serial.println(who, HEX);
  } else {
    Serial.println("WHO_AM_I read failed (check wiring / address)");
  }
}
int16_t read16(uint8_t reg) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, (uint8_t)2);
  // wait for 2 bytes
  unsigned long t0 = millis();
  while (Wire.available() < 2) {
    if (millis() - t0 > 50) return 0; // timeout protection
  }
  uint8_t hi = Wire.read();
  uint8_t lo = Wire.read();
  return (int16_t)((hi << 8) | lo);
}
void loop() {
  int16_t ax = read16(0x3B);
  int16_t ay = read16(0x3D);
  int16_t az = read16(0x3F);
  int16_t gx = read16(0x43);
  int16_t gy = read16(0x45);
  int16_t gz = read16(0x47);
```

```
  float accelX = ax / 16384.0;
  float accelY = ay / 16384.0;
  float accelZ = az / 16384.0;
  float gyroX  = gx / 131.0;
  float gyroY  = gy / 131.0;
  float gyroZ  = gz / 131.0;
  // print comma separated floats, newline at end
  Serial.print(accelX, 4); Serial.print(",");
  Serial.print(accelY, 4); Serial.print(",");
  Serial.print(accelZ, 4); Serial.print(",");
  Serial.print(gyroX , 4); Serial.print(",");
  Serial.print(gyroY , 4); Serial.print(",");
  Serial.println(gyroZ , 4);
  delay(50); // 20 Hz
}
```

Note: The above code to be uploaded in ESP32C6 Dev board using Arduino IDE to read the angle.

```
import processing.serial.*;
import java.util.ArrayList;
Serial port;
float x, y;    // current position
float px, py;  // previous position
float sensitivity = 0.5;
ArrayList<Particle> particles = new ArrayList<Particle>();
color[] palette = {#FF0055, #00E5FF, #00FF66, #FFD500, #A100FF};
// Gradient background colors
color bgTop = color(70, 0, 140);
color bgBottom = color(0, 120, 255);
void setup() {
  size(900, 700);
  drawGradientBackground();
  drawTitle();      // draw “BUILD CLUB” text once
  smooth(8);
  println("Available serial ports:");
  println(Serial.list());
  port = new Serial(this, "COMXX", 115200);
  port.bufferUntil('\n');
  x = width/2;
  y = height/2;
  px = x;
  py = y;
}
void draw() {
```

```
  // translucent overlay for trails
  noStroke();
  fill(0, 25);
  rect(0, 0, width, height);
  // Redraw the title softly so it glows behind effects
  drawTitleGlow();
  // Draw all moving particles
  for (int i = particles.size()-1; i >= 0; i--) {
    Particle p = particles.get(i);
    p.update();
    p.display();
    if (p.dead()) particles.remove(i);
  }
  // Draw motion line
  stroke(255, 180);
  strokeWeight(1.5);
  line(px, py, x, y);
  px = x;
  py = y;
}
void serialEvent(Serial p) {
  String line = trim(p.readStringUntil('\n'));
  if (line == null || line.length() == 0) return;
  if (!Character.isDigit(line.charAt(0)) && line.charAt(0) != '-') return;
  String[] v = split(line, ',');
  if (v.length < 6) return;
```

```
  float gyroX = float(v[3]);
  float gyroY = float(v[4]);
  // Move cursor based on gyro
  x += gyroY * sensitivity;
  y -= gyroX * sensitivity;
  x = constrain(x, 0, width);
  y = constrain(y, 0, height);
  // Spawn particles based on motion speed
  float speed = dist(px, py, x, y);
  int count = int(map(speed, 0, 30, 1, 8));
  for (int i = 0; i < count; i++) {
    particles.add(new Particle(x, y, palette[int(random(palette.length))]));
  }
}
void keyPressed() {
  if (key == 'c' || key == 'C') {
    drawGradientBackground();
    drawTitle();
    particles.clear();
  }
}
// -----------------------
// Particle Class
// -----------------------
class Particle {
  float x, y, vx, vy, alpha, size;
```

```
color c;
Particle(float x_, float y_, color c_) {
  x = x_;
  y = y_;
  c = c_;
  float angle = random(TWO_PI);
  float speed = random(0.5, 2.5);
  vx = cos(angle) * speed;
  vy = sin(angle) * speed;
  alpha = 255;
  size = random(5, 15);
}
void update() {
  x += vx;
  y += vy;
  vx *= 0.98;
  vy *= 0.98;
  alpha -= 4;
}
void display() {
  noStroke();
  fill(c, alpha);
  ellipse(x, y, size, size);
}
boolean dead() {
  return alpha <= 0;
```

```
  }
}
// -----------------------
// Background Gradient
// -----------------------
void drawGradientBackground() {
  for (int i = 0; i < height; i++) {
    float inter = map(i, 0, height, 0, 1);
    color c = lerpColor(bgTop, bgBottom, inter);
    stroke(c);
    line(0, i, width, i);
  }
}
// -----------------------
// “BUILD CLUB” Title Glow
// -----------------------
void drawTitle() {
  textAlign(CENTER, CENTER);
  textSize(100);
  PFont font = createFont("Arial Bold", 100);
  textFont(font);
  // glow layers
  fill(255, 255, 255, 30);
  for (int i = 12; i > 0; i -= 2) {
    textSize(100 + i);
    fill(0, 200, 255, 20);
```

```
    text("BUILD CLUB", width/2, height/2);
  }
  // main text
  fill(255);
  textSize(100);
  text("BUILD CLUB", width/2, height/2);
}
void drawTitleGlow() {
  textAlign(CENTER, CENTER);
  textSize(100);
  fill(0, 200, 255, 10);
  text("BUILD CLUB", width/2, height/2);
}
```

Note: The above code to be uploaded in Processing Software, while uploading please close the Serial Monitor in Arduino IDE, then only the project works.

- While uploading please verify the COM port in your laptop and mention that COM port in the above code.

Tasks:

1) Modify the visuals by changing colours, gradients, or particle effects and observe how it affects user experience.
2) Test different orientations of the MPU6050 sensor and adjust the code to map gestures correctly.
3) Experiment with shapes or effects in the visuals that respond to hand speed or direction.
4) Measure the response delay between gestures and on-screen visuals and optimize it for smoother control.